

National Autonomous University of Mexico

Faculty of Engineering

Computer Engineering

Compilers

JACARANDA
PARSER & SDT REPORT

Team 01 – Group 5

Semester 2027-1

Team members: Aguilar Lara Ericka – 321227122
Bautista Reyes Sofía – 321103891
Cruz Basilio Ximena Carolina – 321116424
Garibay Zamorano Josué Benjamín – 321206105
Jardón Marín Rodrigo – 321240693

Mexico City, Mexico. November 2026

Contents

1 Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Motivation . . . 2
- 1.3 Objectives . . . 2

2 Theoretical Framework **3**
- 2.1 Syntax Analysis and Grammars . . . 3
- 2.2 Top-Down Recursive Descent . . . 3
- 2.3 Parse Tree and AST . . . 3
- 2.4 Syntax-Directed Translation . . . 3
- 2.5 Error Categories . . . 4

3 Development **4**
- 3.1 Project Architecture . . . 4
- 3.2 Grammar and Supported Structures . . . 6
  - 3.2.1 Grammar Implementation . . . 6
- 3.3 Core Parsing Diagrams . . . 7
- 3.4 Parser Decision Diagrams . . . 9
- 3.5 Functions and Nested Blocks . . . 9
- 3.6 Conditional Statements and Loops . . . 11
- 3.7 Tree Visualization . . . 11
- 3.8 Expression Recognition: Basic and Extended Rules . . . 11
- 3.9 Semantic Verification Procedure . . . 14
- 3.10 Testing and Evidence Method . . . 16

4 Results **16**
- 4.1 Basic Syntax and SDT Validation . . . 16
- 4.2 Function and Return Statement . . . 16
- 4.3 Parse Tree of a Function . . . 17
- 4.4 Function Parameters and Calls . . . 18
- 4.5 Conditionals and Repetition . . . 19
- 4.6 Syntactic Error . . . 20
- 4.7 Semantic Error after a Successful Parse . . . 20
- 4.8 Duplicate Declaration and Function-Call Error . . . 21
- 4.9 Selected Test Summary . . . 23

5 Repository **23**

6 Conclusions **23**

1 Introduction

1.1 Problem Statement

During the first delivery, Jacaranda recognized lexical tokens through RPLY. However, a sequence of valid tokens does not necessarily form a valid instruction, and a syntactically correct instruction may still violate semantic rules. For example, an incomplete declaration is a syntax problem, while assigning a Boolean value to an integer variable is a semantic problem under the rules selected for this project.

The second delivery extends the lexical analyzer with a syntax analyzer and syntax-directed semantic verification (SDT). The new implementation recognizes a documented subset of C-like programs, constructs a parse tree and an abstract syntax tree (AST), and reports where a lexical, syntactic or semantic error occurs. It does not execute the analyzed source code or claim complete C compatibility.

1.2 Motivation

The development follows the stages of a compiler: lexical recognition, grammatical validation and semantic checks. Keeping these stages separate makes the analyzer easier to test and explain. The project was extended to include functions, blocks and control-flow statements in order to study how the structure of a larger program is represented and checked, rather than restricting the demonstration to single declarations [1, 2].

1.3 Objectives

General objective. Extend Jacaranda with a manually implemented top-down parser and an SDT module that process tokens from the existing lexer, build syntactic trees and verify selected semantic constraints.

Specific objectives.

- Reuse the RPLY lexer and preserve the distinction between lexical, syntactic and semantic errors.
- Define and implement a grammar for variables, functions, calls, blocks, conditional statements, loops and expressions.
- Build a concrete parse tree and an AST without using a parsing library.
- Check declared identifiers, variable types, duplicate declarations, existing functions and return values.
- Provide terminal and graphical access, including a visual Parse Tree viewer.

- Document reproducible successful and failing cases and collect real application screenshots.

2 Theoretical Framework

2.1 Syntax Analysis and Grammars

A parser verifies whether the token sequence produced by the lexer can be derived from the grammar. Terminals correspond to recognized lexemes and token types, while non-terminals represent grammatical structures such as a function, statement, expression or block. Jacaranda checks both the token kind and, when needed, its textual value. Thus, two words classified as keywords can lead to different parser rules [1].

2.2 Top-Down Recursive Descent

Jacaranda uses a manually written top-down recursive-descent parser. Each principal grammatical structure is recognized by a dedicated parsing procedure. The parser inspects the next token to decide whether it is processing a declaration, function definition, assignment, conditional statement, loop, print instruction or return statement. It uses separate expression levels to enforce operator precedence. The method does not rely on a third-party parsing library.

2.3 Parse Tree and AST

The *parse tree* retains the grammar's non-terminals and recognized terminals, making the derivation visible. The *AST* is a compact representation of the same source that keeps significant operations, declarations and statement relationships. The SDT operates on the AST rather than interpreting the presentation drawing.

The tree viewer shows the parse tree using the Jacaranda interface and supports zoom, navigation and export as PNG, JPG or SVG. The end-of-input marker (EOF) is checked internally but hidden from this visual representation. A large tree may require horizontal and vertical navigation; the exported image contains the full tree.

2.4 Syntax-Directed Translation

SDT attaches semantic checks to the structures recognized by the parser. Jacaranda synthesizes an expression type from literals, identifiers, function calls and operations, and consults symbol tables for contextual information. It also records declared functions and handles local variable scopes [3, 4].

Table 1: Main semantic rules considered by Jacaranda.

Element	Semantic verification
Variable declaration	Reject a second declaration of the same identifier in the current scope; check the initializer type.
Variable reference	Search from the innermost scope outward; report an undeclared variable when no match exists.
Function definition	Record the function name and return type; reject a repeated definition.
Function call	Check that the function was declared; do not compare argument types or counts with its parameters.
Return statement	Check that the value is compatible with the enclosing function's declared return type.
Conditions	Accept numeric or Boolean expressions in the supported control structures.
Arithmetic and logical operations	Apply operand restrictions and synthesize the result type.

The implementation allows selected widening conversions, such as an integer value assigned to a floating-point variable. As an educational simplification, division yields `float` even for two integers; this does not reproduce the full semantics of C. The analyzer checks expressions and declarations but does not execute them or compute program output.

2.5 Error Categories

A lexical error means the scanner cannot recognize a character or token. A parsing error means the token sequence violates the grammar. An SDT error means the syntax was accepted but the semantic restrictions were not satisfied. These categories are kept separate so that a successful parse can still be followed by an SDT error.

3 Development

3.1 Project Architecture

The lexer is implemented in `LexicalAnalyzer.py` and uses RPLY [5]. The analysis pipeline sends its tokens to the parser, which builds the parse tree and AST. A separate semantic module checks types and scopes. Jacaranda offers both a terminal mode and a Tkinter interface. The latter includes a dedicated parse-tree viewer. No fallback lexer or third-party parsing library is used.

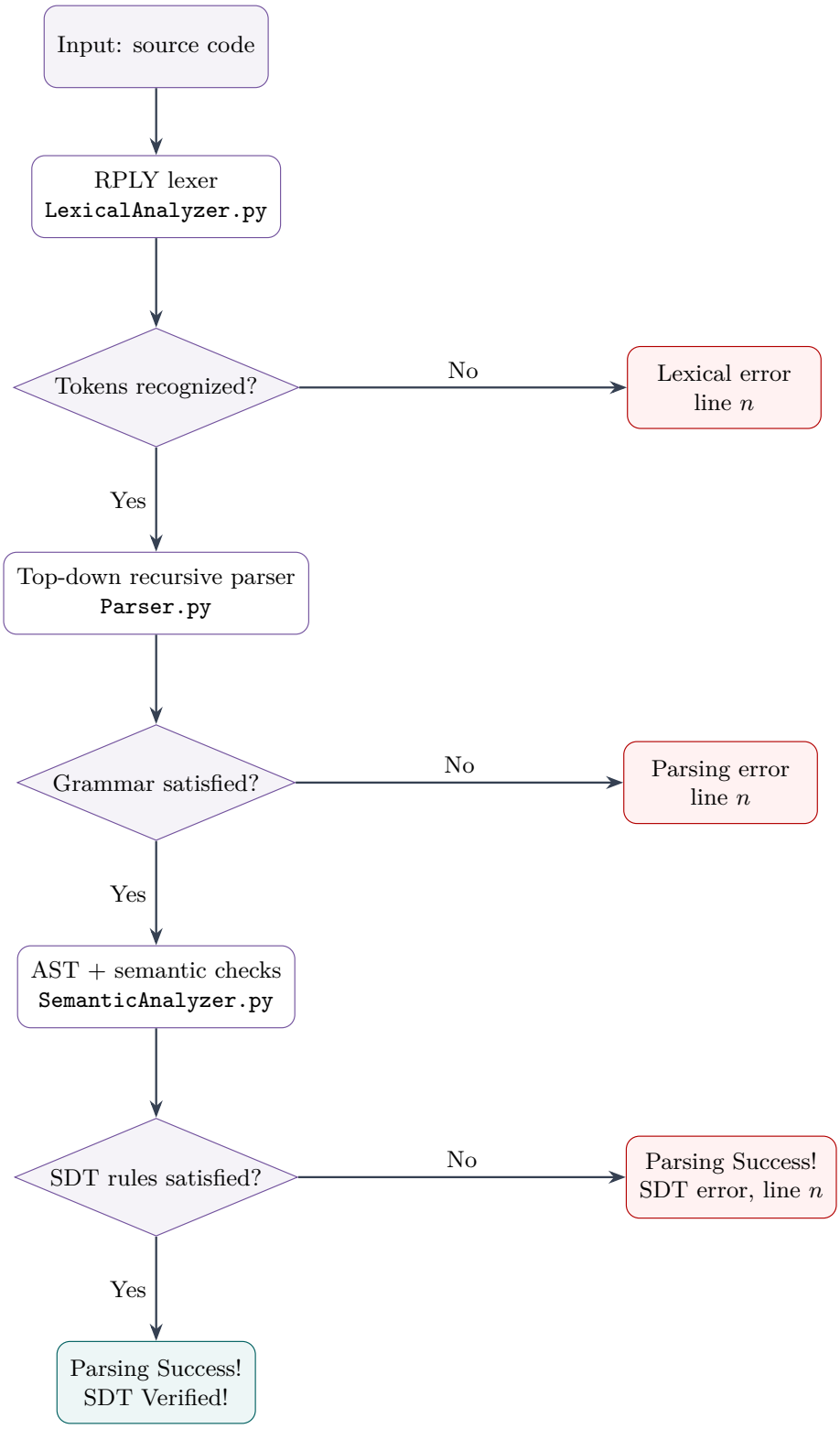


Figure 1: Overall analysis flow and error handling in Jacaranda.

3.2 Grammar and Supported Structures

The grammar is intentionally limited to the subset implemented in the repository. At the program level, the parser accepts global declarations and function definitions; it also retains support for the top-level statements used in the basic test examples. The parsing procedures cover the following constructions:

Table 2: Main groups of productions used in the extended grammar.

Grammar group	Accepted structure
Program / global	Sequences of global declarations and function definitions; a function includes a return type, name, parameters and body.
Block	A brace-delimited sequence of supported statements.
Variables	Typed variables, optional initializers, assignments and increment/decrement statements.
Functions	Typed parameters, prototypes, definitions and calls with argument lists.
Output and return	`print`, simplified `printf`, and return statements.
Control flow	`if`, optional `else`, `while`, `for` and `do-while`.
Expressions	Literals, names, function calls, parentheses, unary operators, arithmetic, comparisons, equality and logical operators.

3.2.1 Grammar Implementation

Jacaranda represents the grammar directly through its parsing functions. Each function recognizes a structure such as a declaration, expression, block or conditional statement. The parser reads tokens from the lexer and selects the appropriate procedure according to their type and value. Operator precedence is handled by separate expression procedures.

A function may have an empty parameter list, a single `void` marker or a comma-separated list of typed parameters. Declarations within blocks follow the same variable structure. The `for` statement separately recognizes an optional initializer, condition and update before its body. Semantic rules reject `void` variable types and verify which expressions and assignments are compatible. These productions describe the supported subset and do not define the complete grammar of C.

The expression parser follows the precedence sequence: logical OR, logical AND, equality, comparison, addition/subtraction, multiplication/division/modulo, unary expressions and primary expressions. Parentheses can group expressions and function calls may appear inside larger expressions. This arrangement prevents, for example, a multiplication from being grouped after a surrounding addition.

3.3 Core Parsing Diagrams

The following diagrams describe how Jacaranda recognizes declarations, assignments and printing instructions. They focus on the core productions; the subsequent diagrams show how statement selection also handles functions, blocks and control-flow structures.

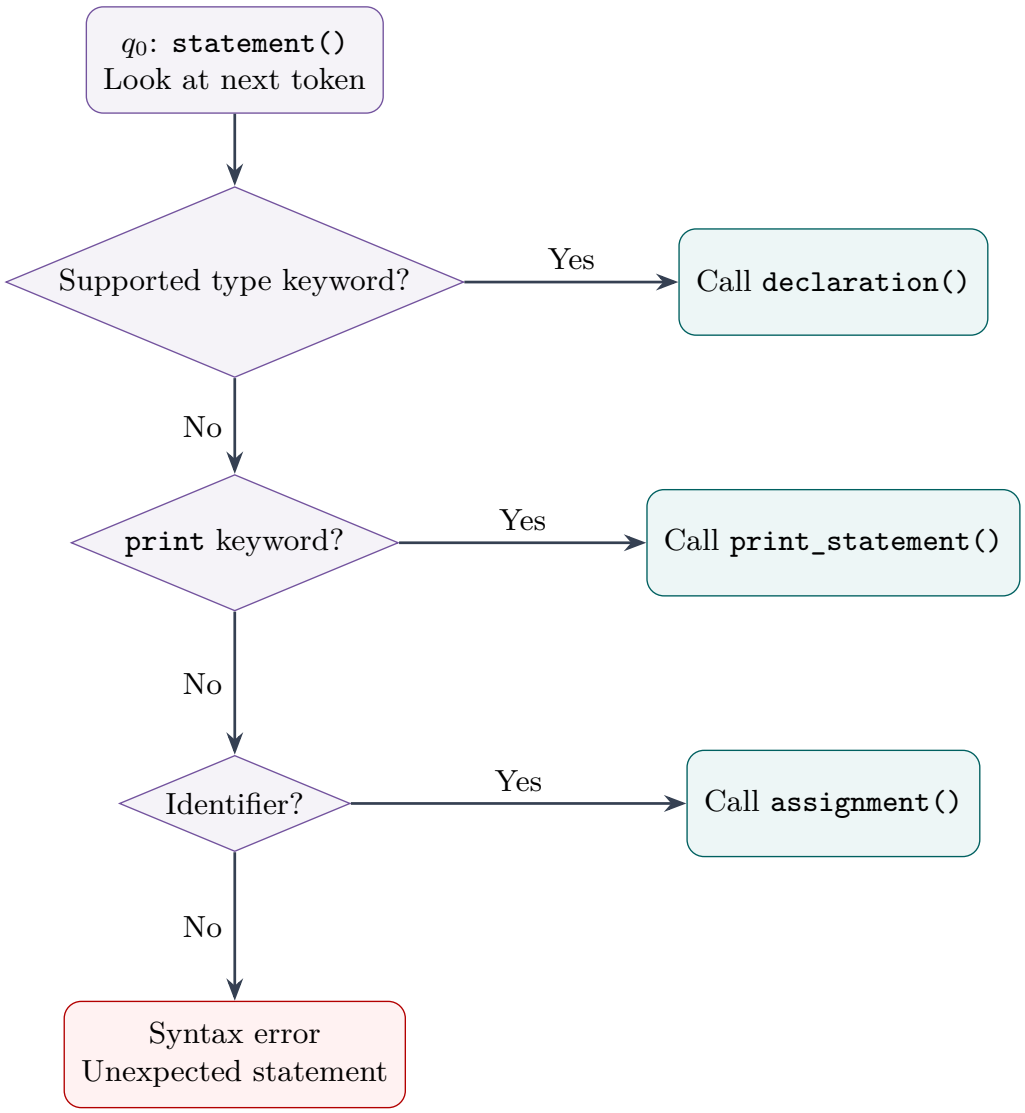


Figure 2: Selection of declaration, assignment and print statements.

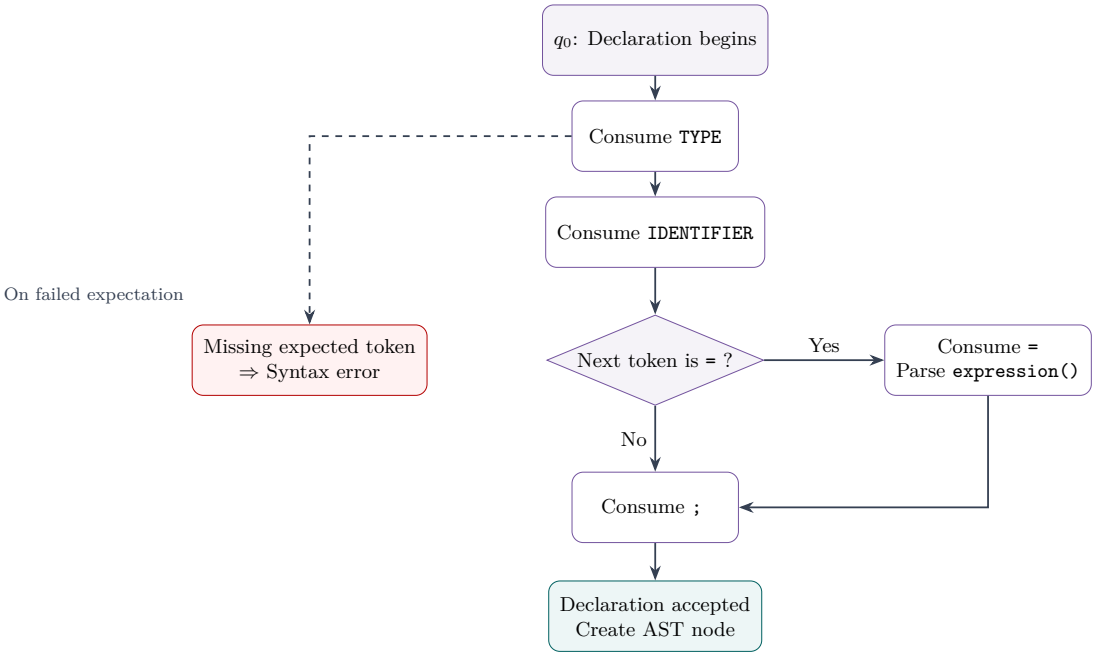


Figure 3: Recognition of a declaration with an optional initializer.

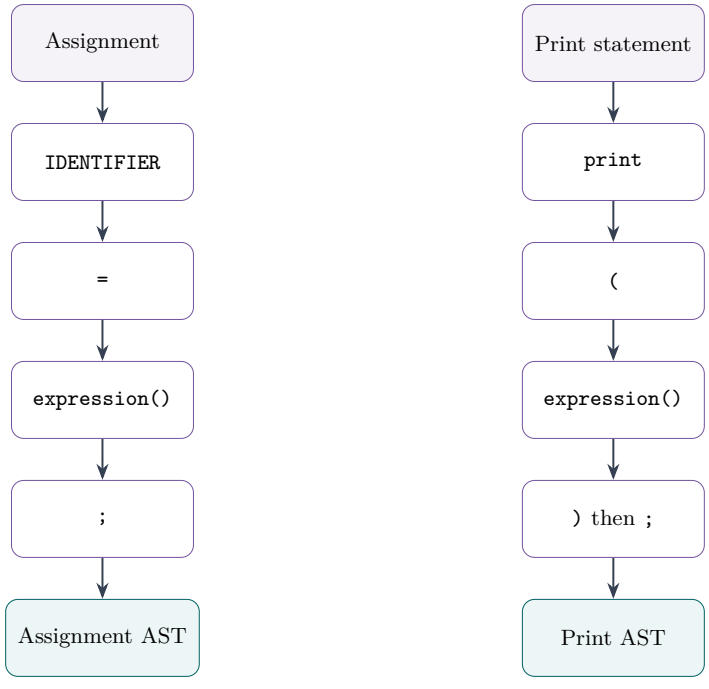


Figure 4: Recognition of assignment and print statements.

3.4 Parser Decision Diagrams

The following diagrams summarize the actual choices made by the recursive-descent parser. They describe the recognition logic; they are not execution traces or lexical finite automata.

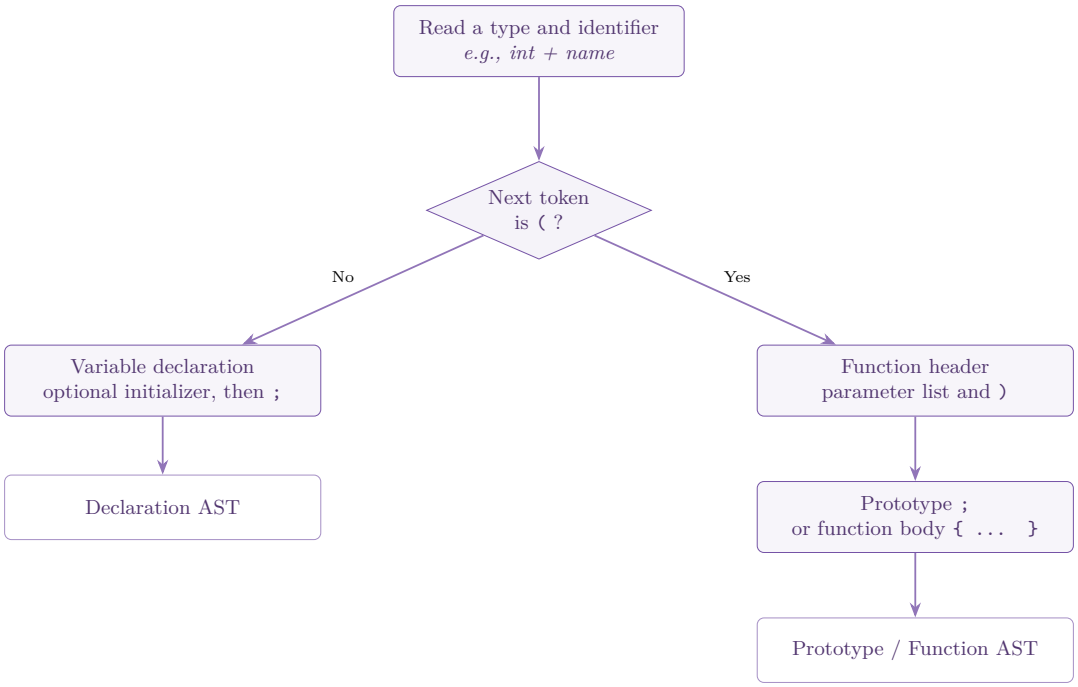


Figure 5: Distinguishing a function header from a global variable declaration after the type and identifier.

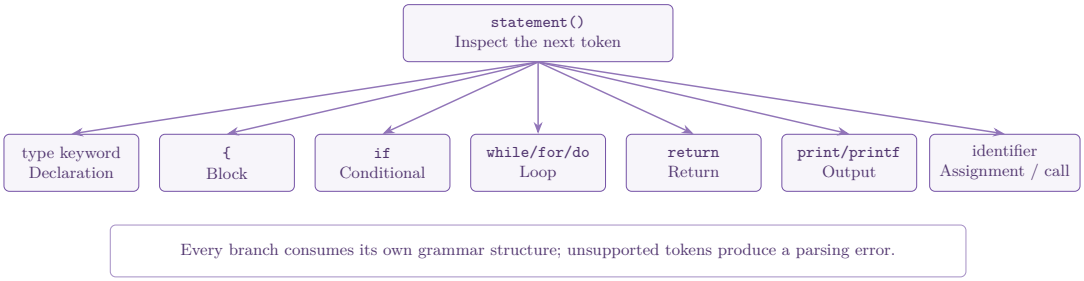


Figure 6: Main groups of statements selected from the next token.

3.5 Functions and Nested Blocks

To recognize a function instead of a variable declaration, the parser examines the token after the type and identifier. An opening parenthesis begins the function path;

otherwise it follows the variable-declaration path. Parameters are recorded in the AST, and the body is analyzed as a block. The SDT first records function names and return types and then verifies the program's statements. This allows calls to functions defined later in the source.

Scopes are represented as nested symbol tables. A new block may introduce local identifiers, while lookup also considers enclosing scopes. A variable declared in a smaller scope does not automatically become available outside it. This is especially important for variables introduced in a `for` loop.

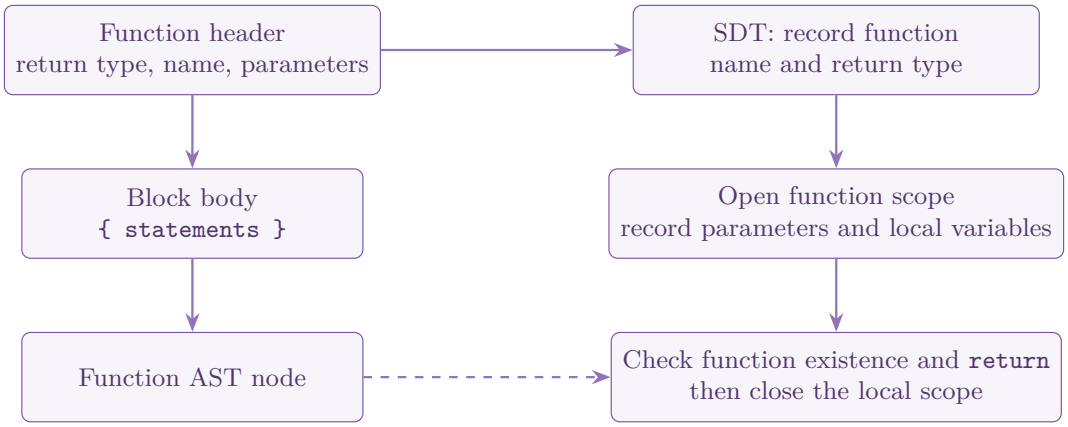


Figure 7: Recognition of a function and the semantic checks associated with its parameters, body and return type.

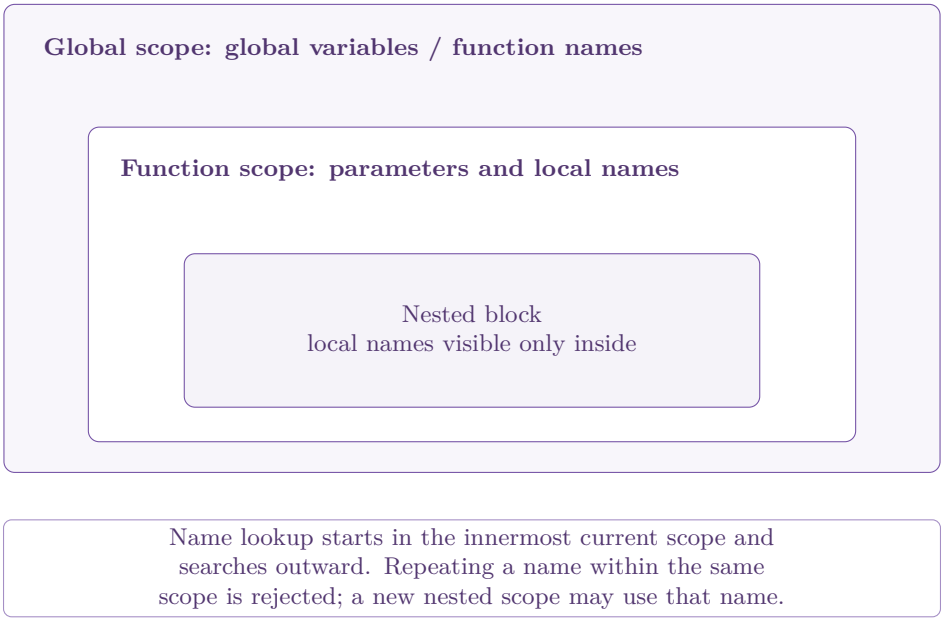


Figure 8: Nested scopes used in variable lookup and duplicate-declaration checks.

3.6 Conditional Statements and Loops

The parser recognizes branches and repetition structures by following their respective grammar rules. The conditions are parsed as expressions and checked by the SDT for allowed numeric or Boolean types. The `for` structure distinguishes initialization, condition, update and body. These constructs are analyzed statically; Jacaranda does not execute a loop or evaluate whether a particular branch will run.

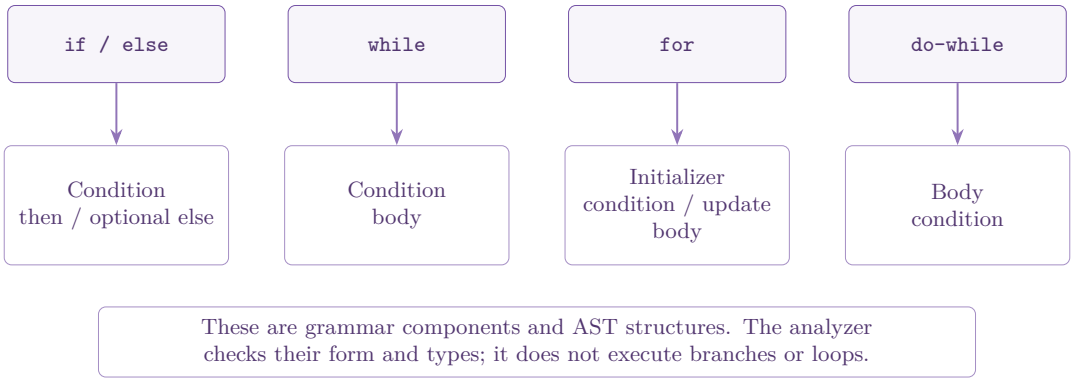


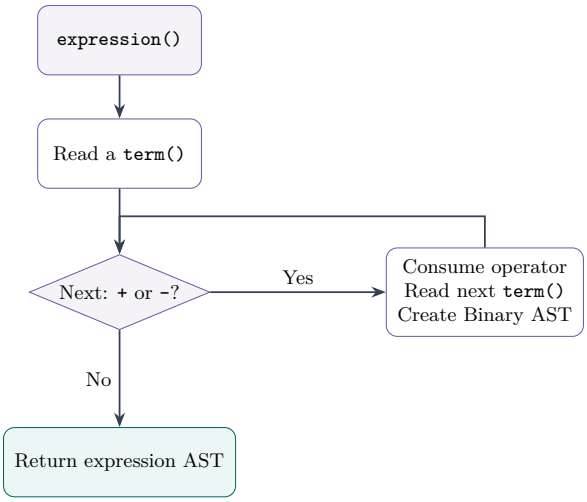
Figure 9: Structural components recognized for conditional and repetition statements; no loop is executed.

3.7 Tree Visualization

The *View Parse Tree* action opens a dedicated Tkinter window using the tree built by the parser [6]. Each branch is positioned using measured label widths and reserved sub-tree space, reducing overlap in larger trees. Internal grammar names remain visible because the purpose of this view is to show the actual derivation. The drawing can be navigated and exported, but exporting is not part of parsing or semantic verification.

3.8 Expression Recognition: Basic and Extended Rules

The expression parser first processes logical and comparison operators before reaching additive and multiplicative operations. The diagrams for arithmetic and primary expressions describe the lower levels of this same grammar. The arithmetic and primary-expression diagrams show the core recognition paths, while Figure 12 presents all expression-precedence levels.



`term()`: the same loop, using `*` and `/`, calls `unary()`.

`unary()`: optional sign; then `primary()` (literal, identifier, parenthesized expression).

Figure 10: Recognition of additive and multiplicative expressions.

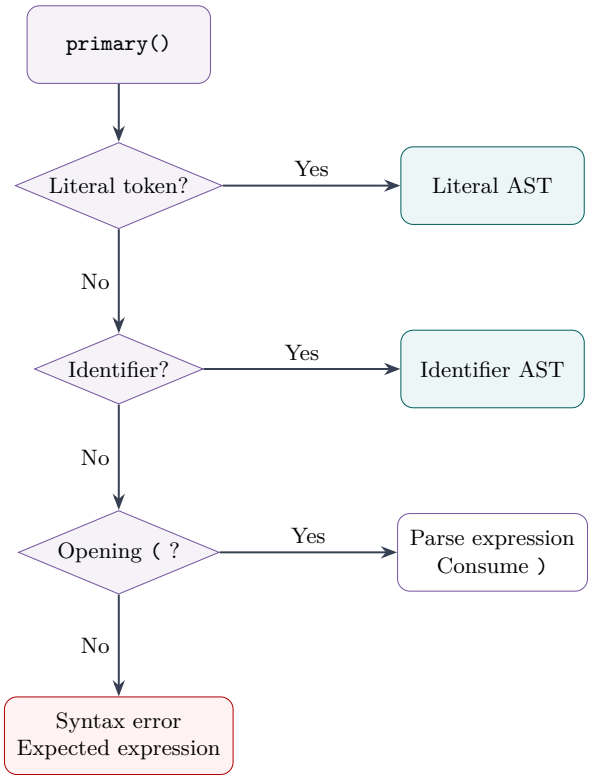


Figure 11: Recognition of literals, identifiers and parenthesized expressions.

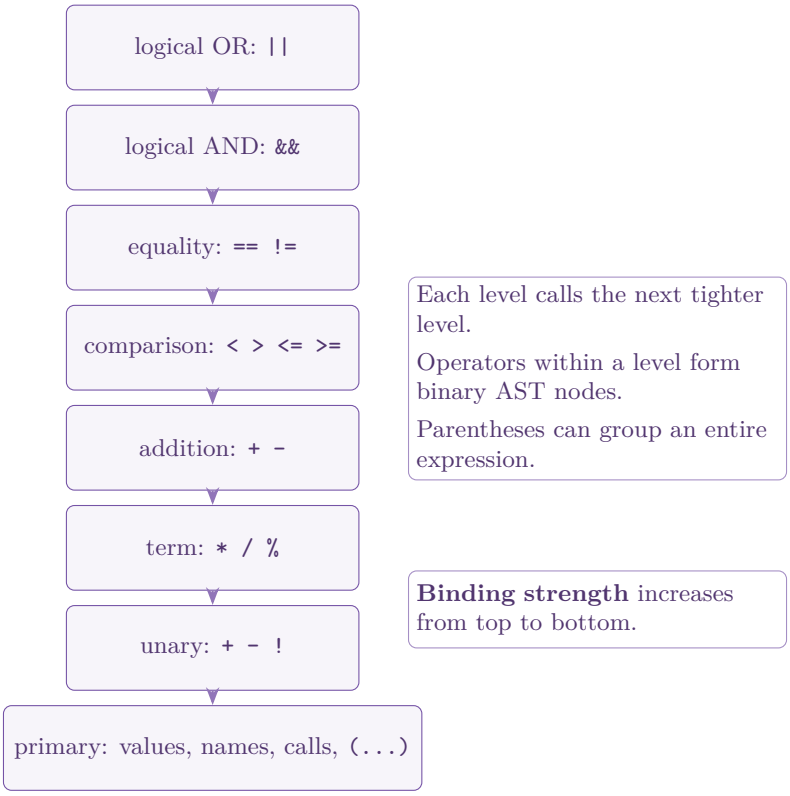


Figure 12: Expression precedence levels used by the recursive-descent parser, from lower binding strength to higher.

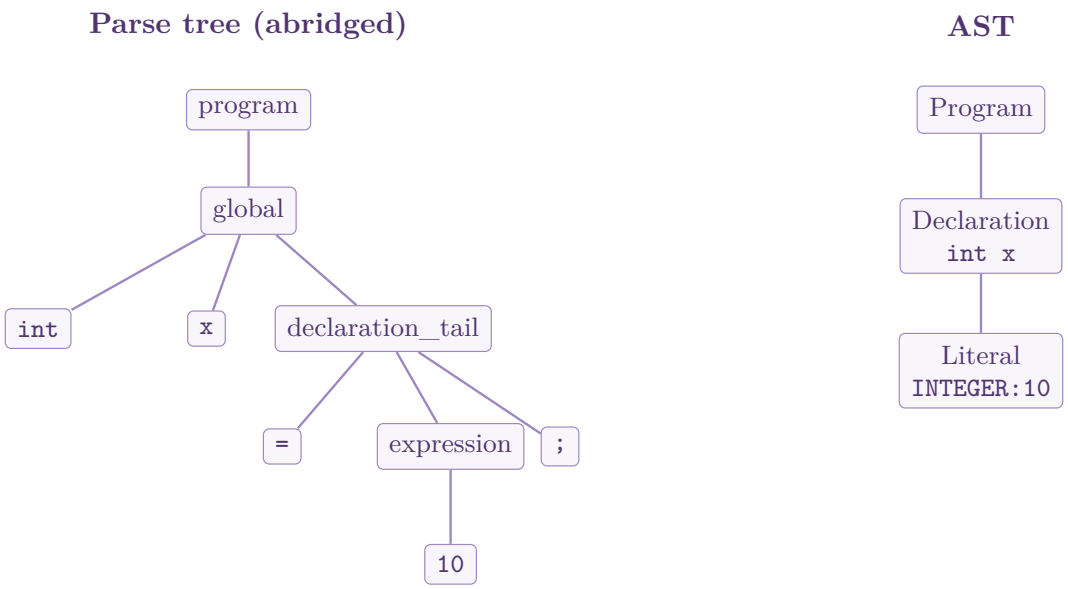


Figure 13: Schematic comparison of an abridged parse tree and an AST for a simple initialized declaration. The actual parse tree contains additional expression levels.

3.9 Semantic Verification Procedure

The semantic analyzer checks each statement's node type. For a variable declaration it validates the initializer before inserting the new symbol. For a function it registers parameter names in the local context, checks function calls and return types, and restores the previous scope when leaving a block. Duplicate declarations in the same scope, unknown names, incompatible types and calls to undeclared functions are reported as semantic errors. Argument counts and types are not compared with a function's parameters.

The `printf` rule is simplified: it requires a string as its first argument but does not validate format specifiers against subsequent values. The parser also does not model pointers, arrays, structures, complete C conversions or the full C standard library. These boundaries distinguish the supported educational subset from a complete C compiler.

The semantic checks cover both individual statements and relationships between functions, parameters and local scopes. Figures 14 and 15 illustrate declaration and assignment checks, while Figure 16 summarizes the complete verification process.

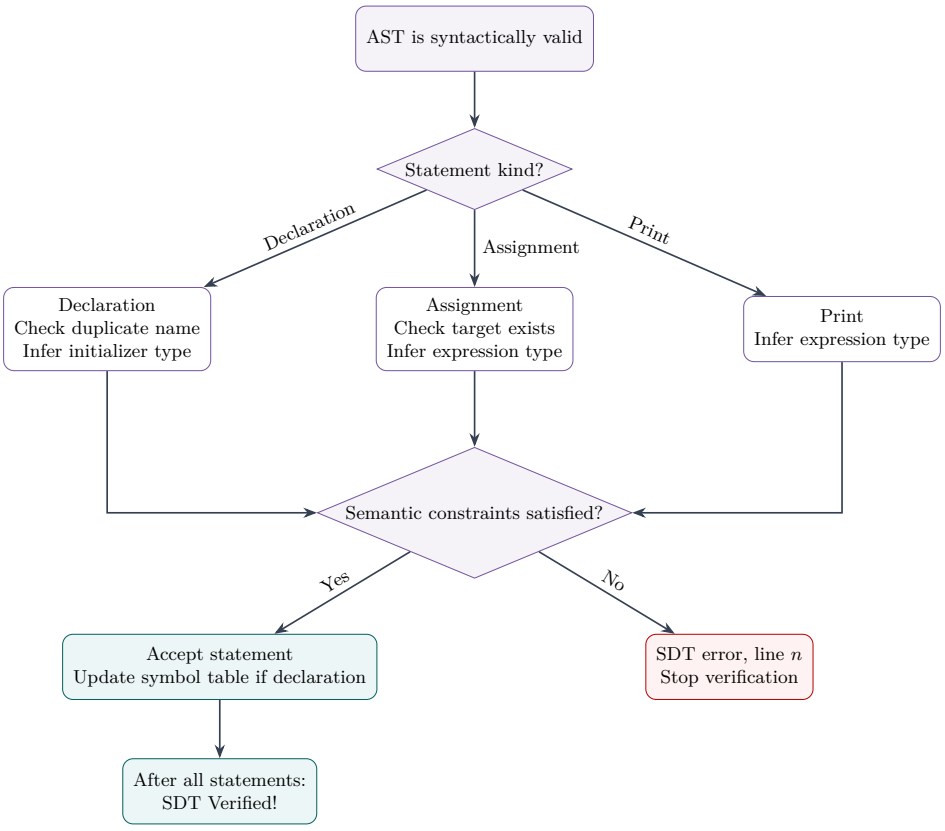


Figure 14: Semantic checks for declarations, assignments and printing.

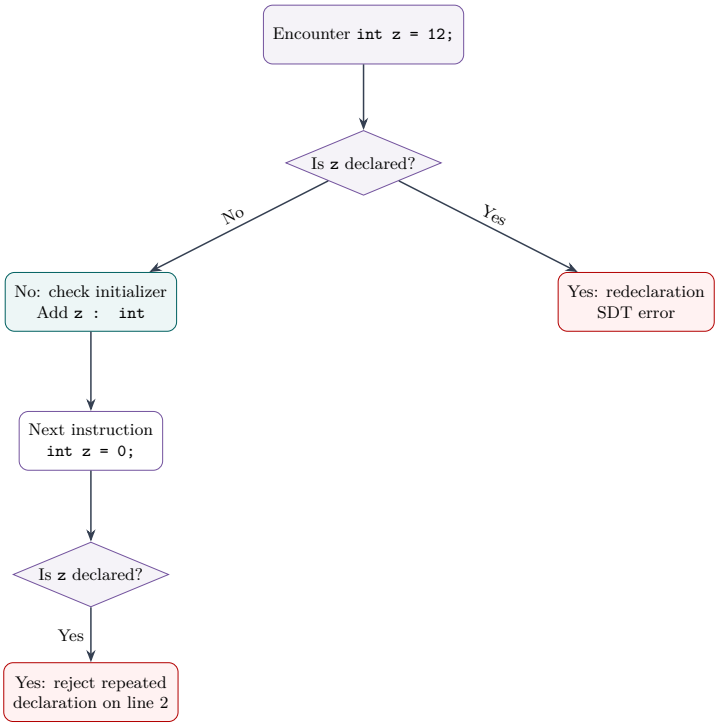


Figure 15: Detection of a repeated declaration in the same scope.

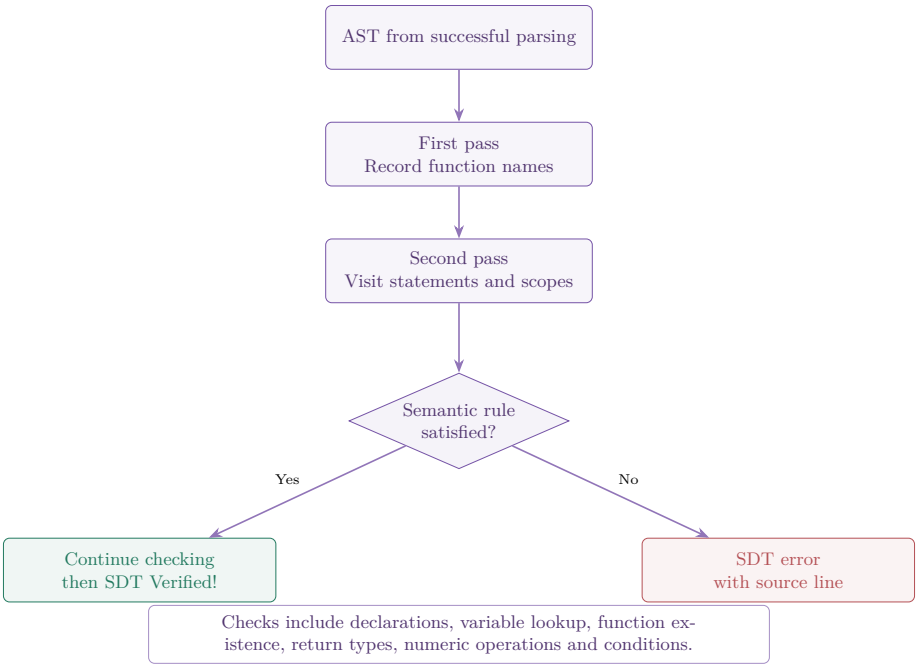


Figure 16: Semantic verification: function names and return types are registered before statements are checked.

3.10 Testing and Evidence Method

The project includes 34 numbered test files in `examples/parser_sdt/`, covering valid programs as well as lexical, syntactic and semantic errors. The suite checks declarations, expressions, functions, blocks, control structures, scopes, return types and undeclared function calls. Automated unit tests complement these example inputs [7].

4 Results

The following representative cases demonstrate the main analysis paths and the distinction between successful parsing, syntax errors and semantic errors.

4.1 Basic Syntax and SDT Validation

The basic declaration case checks that an integer initialization is accepted and that the semantic phase completes successfully. The result displays both parsing success and SDT verification.

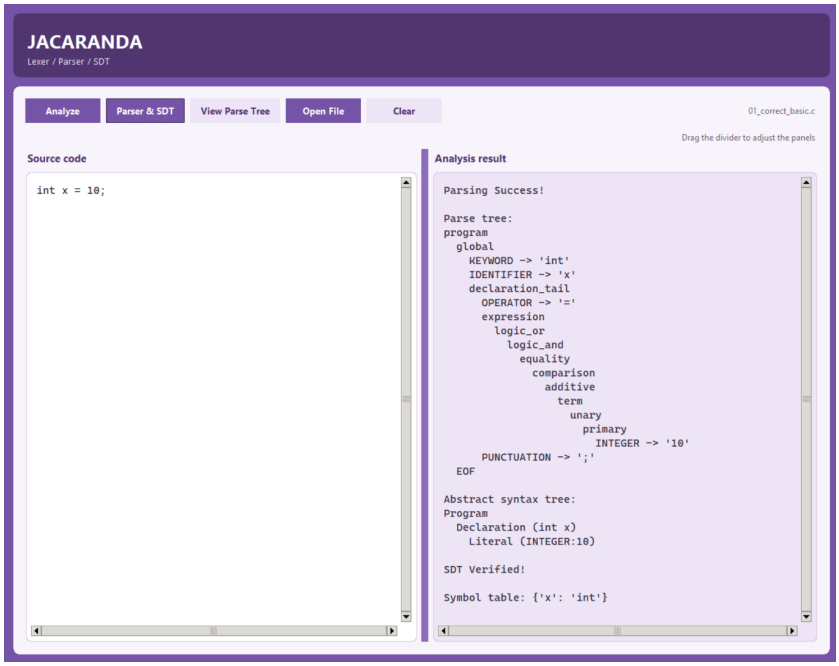


Figure 17: Execution of the valid basic declaration (example 01).

4.2 Function and Return Statement

The function example checks recognition of `main`, a block, a printing statement and a return value. It also provides a clear case for explaining the difference between

grammatical structure and program execution.

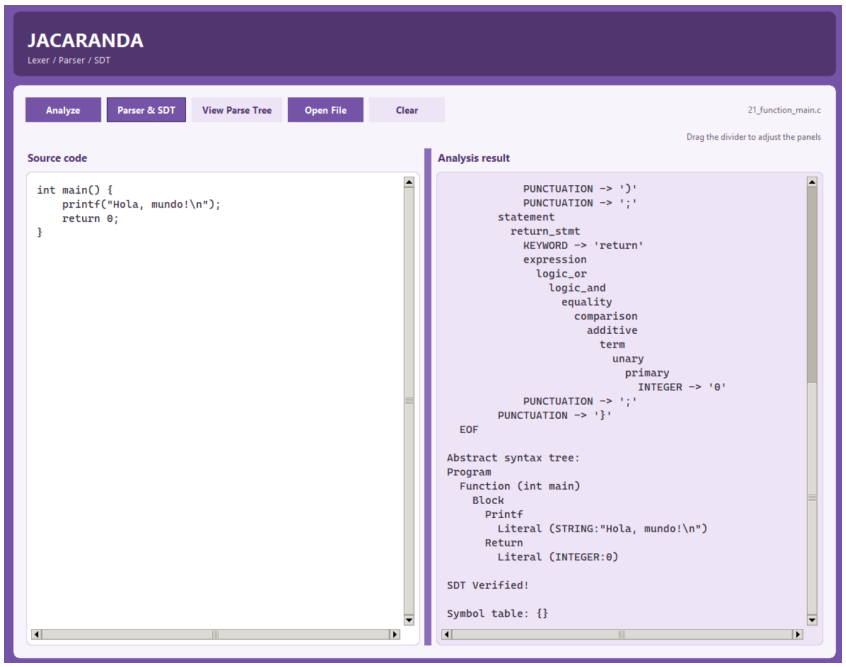


Figure 18: Successful analysis of a function body and return statement (example 21).

4.3 Parse Tree of a Function

The parse tree for the function contains the program root, function header, block and statement branches. The diagram makes the grammatical hierarchy visible, including the printing instruction and return statement.

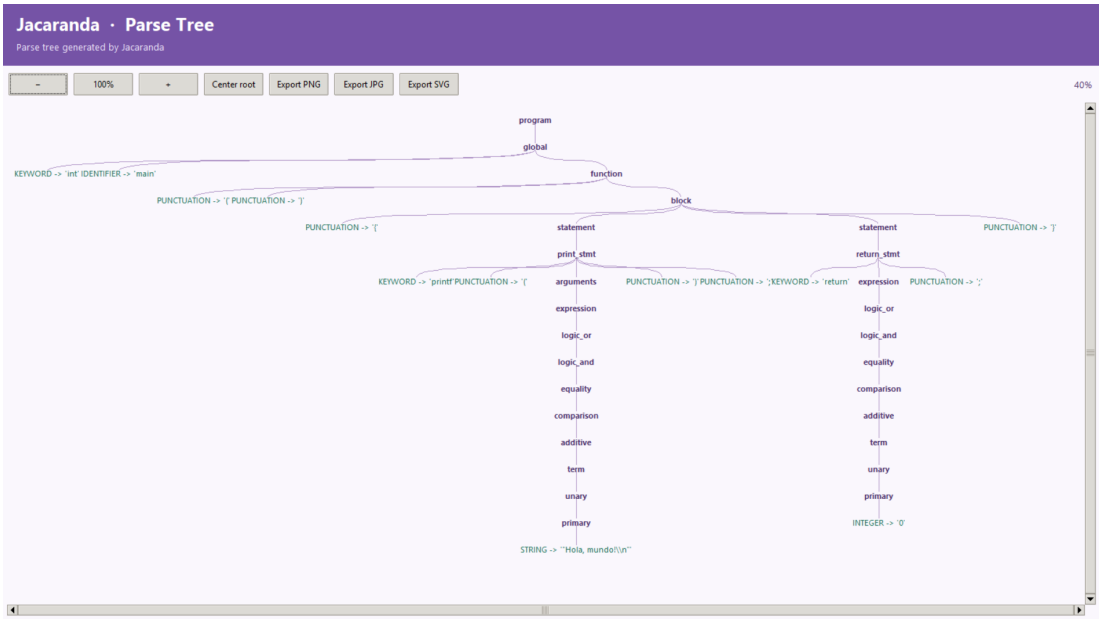


Figure 19: Parse Tree visualization for the function example.

4.4 Function Parameters and Calls

The function-call case checks a declaration with two typed parameters and a subsequent call using arguments. It illustrates syntactic recognition of a function and semantic use of its declared name and return type.

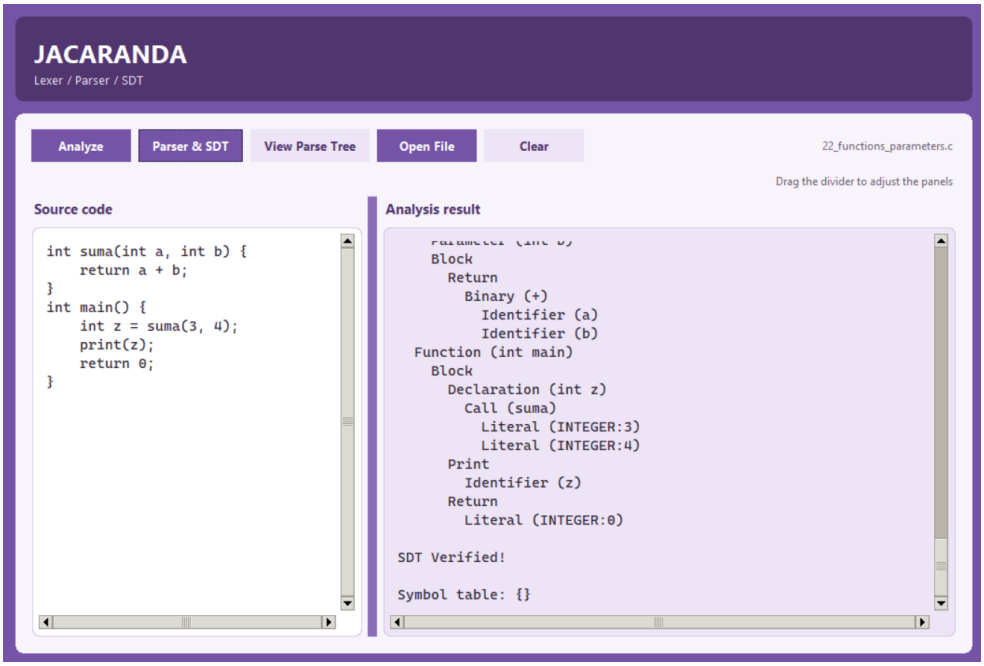


Figure 20: Successful function definition and call (example 22).

4.5 Conditionals and Repetition

The conditionals and loops tests verify the additional grammar branches. The selected examples demonstrate a conditional with an optional alternate branch and a `for` statement with initialization, condition and update.

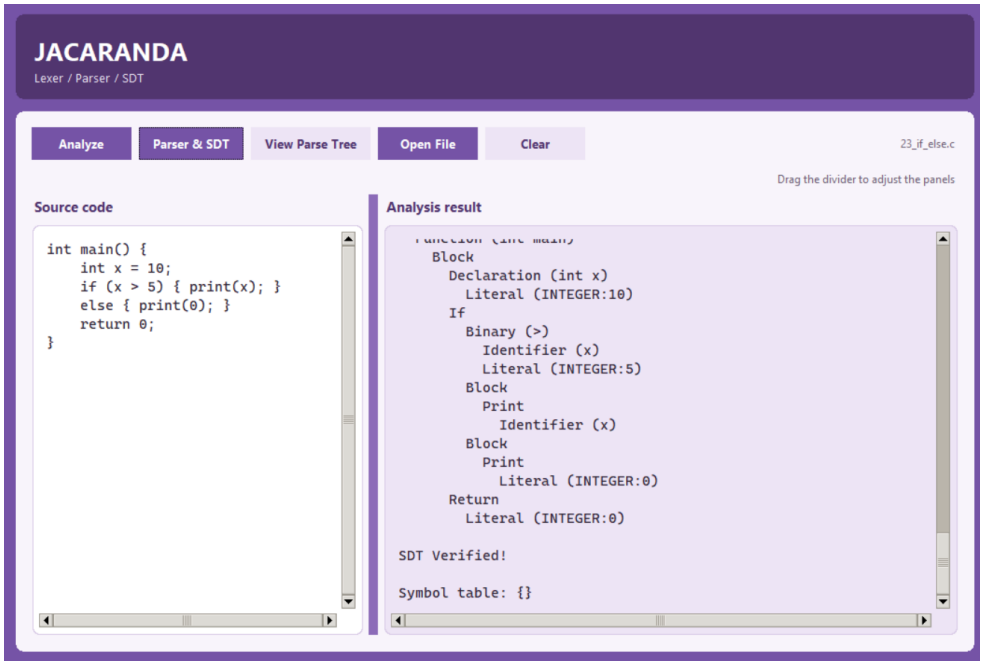


Figure 21: Conditional statement recognized by the parser (example 23).

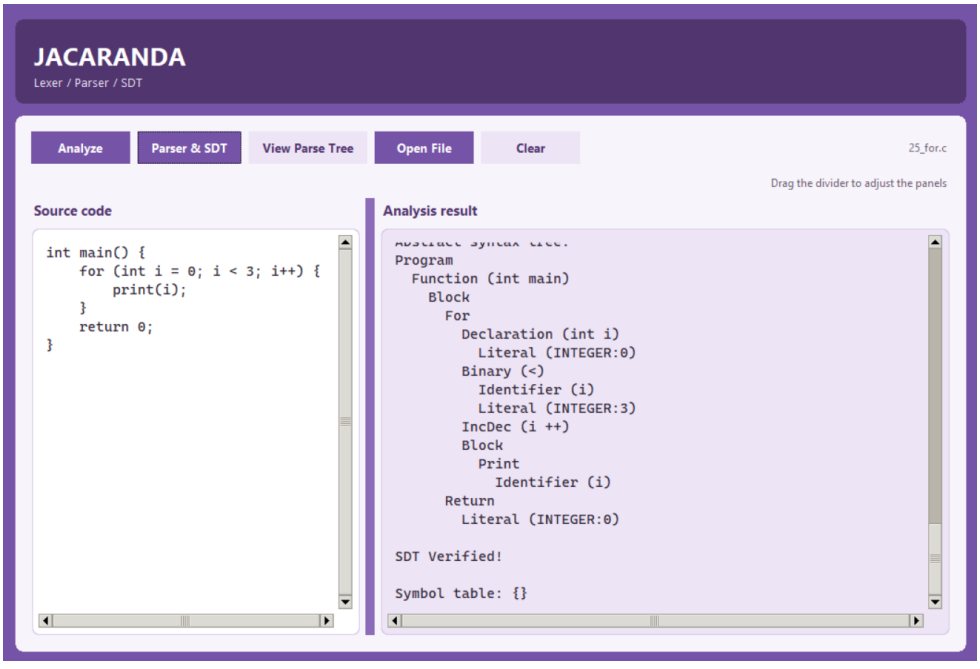


Figure 22: For-loop structure recognized by the parser (example 25).

4.6 Syntactic Error

In the incomplete declaration example, the parser reaches an assignment operator but does not find an expression. Lexical analysis succeeds, while syntactic analysis reports the unexpected token or missing expression. The SDT does not run after this failure.

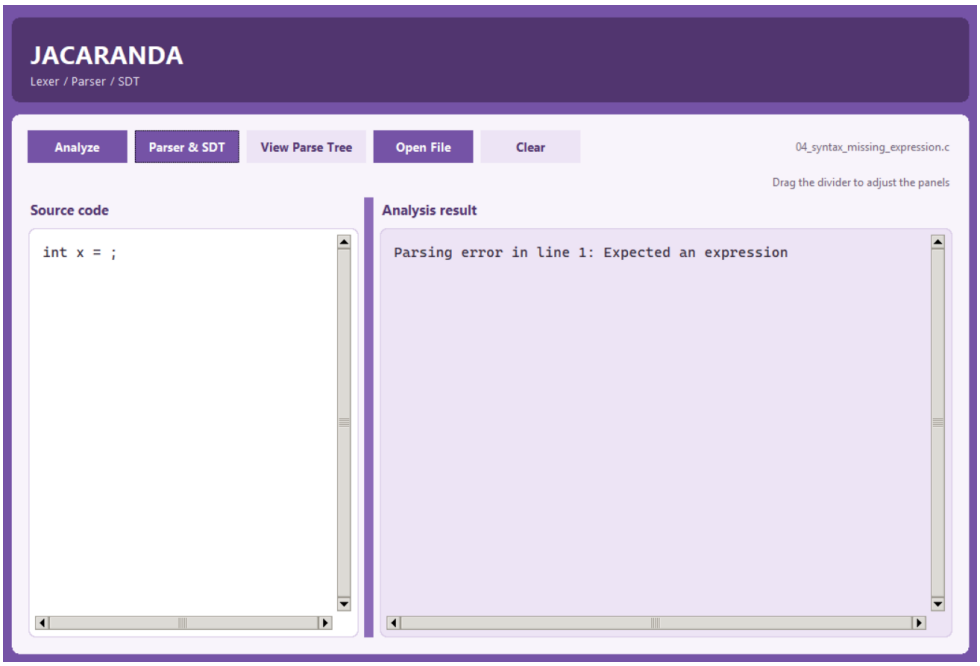


Figure 23: Parsing error for an incomplete declaration (example 04).

4.7 Semantic Error after a Successful Parse

In the type-mismatch example, the declaration follows the grammar, but the initializer is a Boolean where an integer is required by Jacaranda's semantic rules. The result shows a successful parse followed by an SDT error.

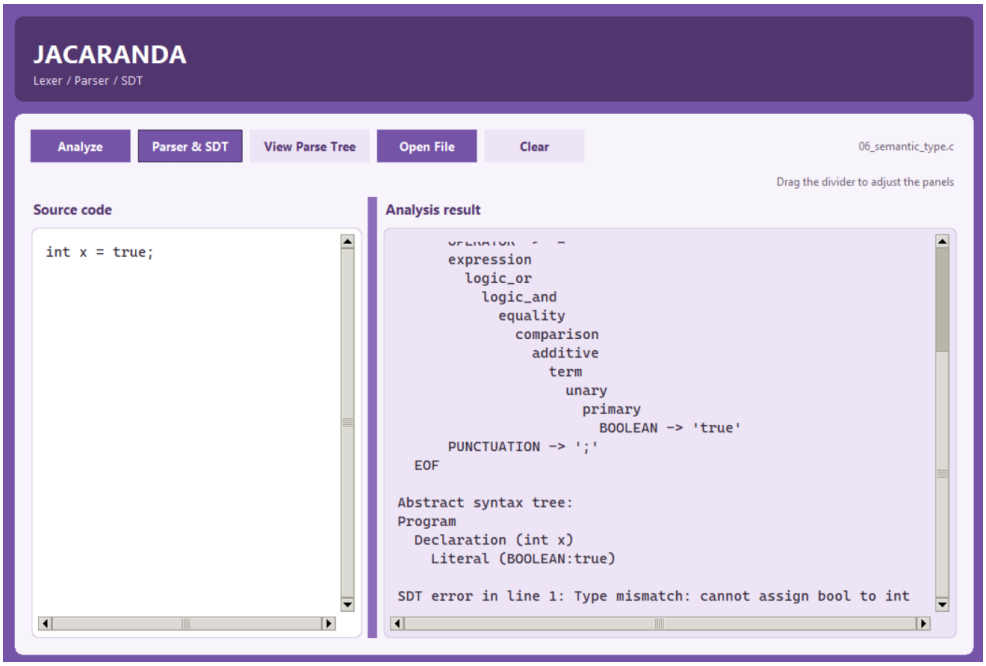


Figure 24: Successful parsing followed by an SDT type error (example 06).

4.8 Duplicate Declaration and Function-Call Error

Two further tests show checks made by the semantic analyzer. One uses the same variable name twice in a block, while the other calls a function that has not been declared.

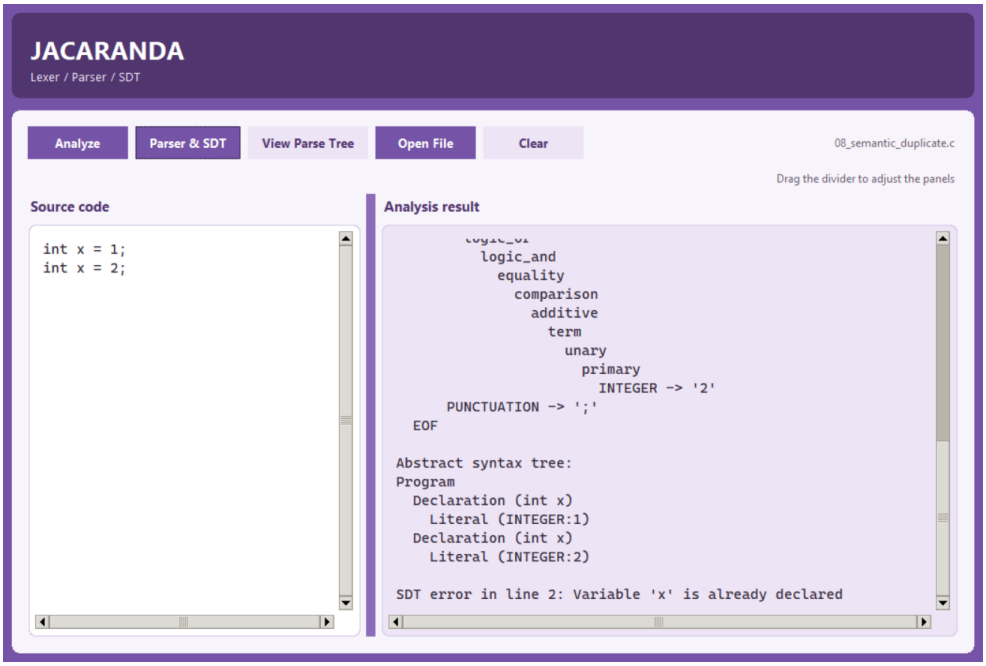


Figure 25: Duplicate declaration detected by the symbol table (example 08).

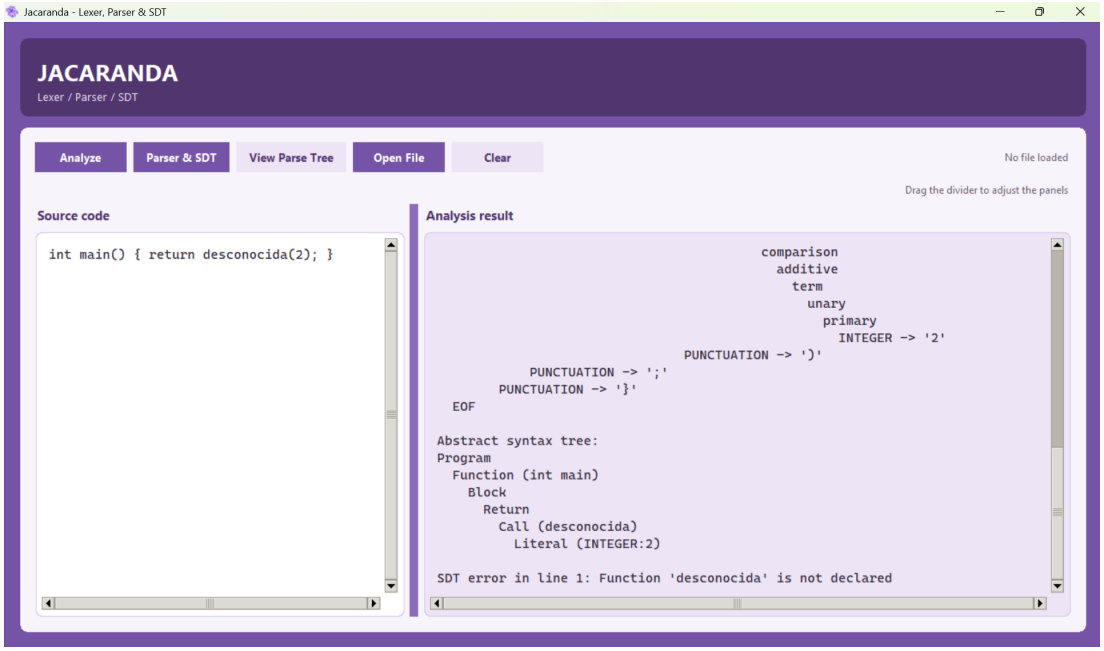


Figure 26: Undeclared function detected by semantic analysis (example 34).

4.9 Selected Test Summary

Table 3: Representative files and expected categories of results.

File in examples/parser_sdt/	Expected behavior
01_correct_basic.c	Parsing success and SDT verified.
21_function_main.c	Function and return accepted.
22_functions_parameters.c	Parameters and a valid call accepted.
23_if_else.c	Conditional with alternative accepted.
25_for.c	Loop recognized and SDT verified.
04_syntax_missing_expression.c	Parsing error.
06_semantic_type.c	Parsing success followed by SDT error.
08_semantic_duplicate.c	Duplicate variable declaration error.
29_valid_function_call.c	Valid function call.
34_semantic_undeclared_function.c	Semantic error for an undeclared function.

The remaining cases cover additional valid constructions, lexical errors, undeclared names, type mismatches and calls to undeclared functions.

5 Repository

Project repository: github.com/ErickaAgui/unam.fi.compilers.g5.01.

6 Conclusions

Aguilar Lara Ericka. This second stage of Jacaranda helped me understand what happens after the lexer identifies the tokens. At first, I thought that recognizing each token was the most important part, but the examples showed me that a program can have valid tokens and still contain syntax or semantic errors. I also learned why it is useful to keep the parser and semantic analyzer separate, since it makes the results easier to understand and review. On the team side, coordinating changes and checking that the different files worked together was just as important as the implementation itself. Seeing the lexer, parser and SDT connected in one application helped me understand the process much better.

Bautista Reyes Sofía. In the first part of the project, I focused on the relationship between automata and regular expressions. With the Parser & SDT, what helped me most was seeing how the tokens are organized into grammatical structures. Looking at the parse tree for a declaration or a function made it easier to understand the order in which the parser recognizes the code. I also understood why an instruction can be

correct in terms of syntax but still have a semantic error, such as assigning a Boolean value to an integer variable. Comparing the parse tree with the simpler AST helped me see why both representations are useful. This stage made the grammar we studied in class easier to connect with the actual results of Jacaranda.

Cruz Basilio Ximena Carolina. Working with the parser diagrams helped me understand the grammar in a more practical way. I had to look at how the parser decides whether it is reading a declaration, a function or another type of statement, instead of only seeing the rules written down. I also paid more attention to the order of expressions and to how blocks are recognized. One thing I learned was that a diagram has to match what the code really does; otherwise, it can make the explanation more confusing. Comparing the diagrams with the parse tree generated by Jacaranda made it easier for me to follow the analysis step by step. Overall, this part helped me understand syntax analysis better than studying the rules alone.

Jardón Marín Rodrigo. This part of the project helped me understand the logic behind a syntax analyzer and how it connects to semantic analysis. The parser has to recognize different instructions while following the grammar, and the semantic analyzer then checks things such as variable declarations and type compatibility. Testing correct and incorrect examples was useful because some mistakes are not detected until after the syntax has been accepted. I also saw how a small change in one part of the project can affect the tests or the interface, which is why reviewing changes in GitHub is important. In the end, I was able to relate the grammar and symbol table concepts from class to a program that produces understandable results.

Garibay Zamorano Josué Benjamín. The second part of Jacaranda helped me see the difference between identifying tokens and checking whether those tokens form a valid program. The examples were especially useful because small changes, such as leaving an expression incomplete or declaring a variable twice, produced different kinds of errors. I also found the parse tree helpful for understanding how statements and expressions are organized, rather than only looking at a success or error message. Keeping the examples and documentation organized made the project easier to test and explain. This stage gave me a clearer idea of how lexical, syntax and semantic analysis work together in a compiler.

References

[1] Github,"Parsing Descendente," *Introducción a la Construcción de Compiladores*. [Online]. Available: `https://matcom.github.io/compilers/chap2.html`

[2] Universitat de València, "Tema 4: Análisis sintáctico descendente." [Online]. Available: `https://informatica.uv.es/docencia/iiguia/asignatu/2000/PL/2007/tema4.pdf`

[3] Github, "Análisis Semántico," *Introducción a la Construcción de Compiladores*. [Online]. Available: `https://matcom.github.io/compilers/chap4.html`

[4] Github, "Gramáticas Atributadas," *Introducción a la Construcción de Compiladores*. [Online]. Available: `https://matcom.github.io/compilers/chap5.html`

[5] RPLY, "RPLY Documentation." [Online]. Available: https://rply.readthedocs.io/en/latest/

[6] Python Software Foundation, "tkinter — Interfaz de Python para Tcl/Tk," *Python Documentation*. [Online]. Available: https://docs.python.org/es/3/library/tkinter.html

[7] Python Software Foundation, "unittest — Marco de pruebas unitarias," *Python Documentation*. [Online]. Available: https://docs.python.org/es/3/library/unittest.html